

Week 5

2. UNIX Process Control Examples (I)

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2560 – System Programming [2025F]

Wednesday, October 1st, 2025



University of Windsor

Content

1 Introduction

2 Unix Processes (Review)

Creating a new Process: `fork()`

Terminating a process: `exit()`

`wait()` and `waitpid()`

3 Examples

4 Assignment #1: Overview

Part 0

File I/O Example



pirate.c – Presented by Erik Hunter

Homework → See `pirate.c` and `dictionary.c` for two more examples.

pirate.c – Presented by Erik Hunter

```
//pirate.c
```

```
#include <stdio.h>

int main (void)
{
    FILE *in, *out;

    struct pirate {
        char name[100]; /* real name */
        unsigned long pounds; /* in pounds sterling */
        unsigned int beard_len; /* in inches */
    } p, blackbeard = { "Edward Teach", 950, 48 };

    struct strange {
        int i;
        float j;
    } test;

    out = fopen ("pirate.data", "w");

    if (!out) {
        perror ("fopen");
        return 1;
    }
    /*not casting for the first argument of fwrite....
    if (!fwrite (&blackbeard, sizeof (struct pirate), 1, out)) {
        perror ("fwrite");
        return 1;
    }

    if (fclose (out)) {
        perror ("fclose");
        return 1;
    }

    in = fopen ("pirate.data", "r");
    if (!in) {
        perror ("fopen");
        return 1;
    }
    if (!fread (&p, sizeof (struct pirate), 1, in)) {
        perror ("fread");
        return 1;
    }

    printf ("name=\"%s\" booty=%lu beard_len=%u\n",
           p.name, p.pounds, p.beard_len);
    //line 48-55 was inspired by a student question in winter 2023
    rewind(in);

    int rt=fread(&test, sizeof(struct strange), 1, in);
    //printf("\nrt=%d\n", rt);
    printf ("test.i=%d, test.j=%f\n", test.i, test.j);

    if (fclose (in)) {
        perror ("fclose");
        return 1;
    }

    return 0;
}
```

Part I

Introduction to Processes



Context

Context

- single computer (single CPU)
- under UNIX operating system

It is possible to have several simultaneously executing programs

→ It appears that the executions are done in Parallel!

Multiprogramming:

The CPU executes some instructions of one program **then switches** to another program giving the **illusion** that any program is continuously executing, this scheme is called **time-slicing**.

Creation of a new process

The creation or spawning of new processes is done with two system calls:

`fork()` : duplicates the caller process

`exec()` : replaces the caller process by a new one.

Part I

UNIX Processes



Creating a new Process: `fork()`

`fork()`

Synopsis: `pid_t fork(void);`

When successful, the `fork()` system call:

1. Creates a copy of the caller (parent) process.
2. Returns the `PID` of the newly created process to the parent
3. Returns 0 to the new process (the child).

If not successful, the `fork()` returns -1.

`fork()` is a "**strange**" system call: It is Called by a single process but returns twice, to two different processes.

The child process

A child process has:

1. Its own unique `PID`,
2. A different `PPID`,
3. Its own copy of the parent's data segment and **file descriptors**.

Both parent and children resume the execution after the call to `fork()`.

`fork()` is primarily used in two situations:

- 1 A process wants to execute another program.
- 2 A process has a main task and when necessary, creates a child to handle an operation (Servers).

Part II

Example 1: Simple `fork()`



Both parent and children resume the execution after the call to `fork()`

```
//fork1.c
```

```
#include <unistd.h>
```

```
int main(int argc, char *argv[]){
```

```
    int pid;
```

```
    printf("Only one process\n");
```

```
    pid = fork();
```

```
    if(pid == -1){
```

```
        perror("impossible to fork");
```

```
        exit(1);
```

```
    }
```

```
    if(pid > 0)
```

```
        printf("I am the parent, pid=%d\n", getpid());
```

```
    else
```

```
        if(pid == 0)
```

```
            printf("I am the child, pid=%d\n", getpid());
```

```
    exit(0);
```

```
}
```

Part III

Example 2: Memory in `fork()`



Creating a new Process: `fork()`Terminating a process: `exit()``wait()` and `waitpid()`

The child has its own copy of the parent's data segment

//fork2.c

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int glb = 100; // global variable
int main(){
    int pid;
    int var = 88;
    printf("Before fork\n");
    pid = fork();
    if( pid < 0 ){
        perror("fork");
        exit(1);
    }
    if(pid == 0){ //child
        glb++; var++;
    }
    else{ // parent
        sleep(2);
    }
    printf("pid = %d, glob = %d, var = %d\n", getpid(), glb, var);
    return 0;
}
```

The child has its own copy of the parent's data segment

//fork2.c

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
```

```
int glb = 100; // global variable
```

```
int main(){
```

```
    int pid;
```

```
    int var = 88;
```



```
    printf("Before fork\n");
```

```
    pid = fork();
```

```
    if( pid < 0 ){
```

```
        perror("fork");
```

```
        exit(1);
```

```
    }
```

```
    if(pid == 0){ //child
```

```
        glb++; var++;
```

```
    }
```

```
    else{ // parent
```

```
        sleep(2);
```

```
    }
```

```
    printf("pid = %d, glob = %d, var = %d\n", getpid(), glb, var);
```

```
    Return 0;
```

```
}
```

Process A (parent)

```
+-----+
```

```
| Code (text) | <- shared, read-only
```

```
+-----+
```

```
| Data (globals) | glb = 100
```

```
+-----+
```

```
| Heap | (unused here)
```

```
+-----+
```

```
| Stack | var = 88
```

```
+-----+
```


Creating a new Process: `fork()`Terminating a process: `exit()``wait()` and `waitpid()`

The child has its own copy of the parent's data segment

//fork2.c

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
```

```
int glb = 100; // global variable
```

```
int main(){
```

```
    int pid;
```

```
    int var = 88;
```

```
    printf("Before fork\n");
```



```
    pid = fork();
```

```
    if( pid < 0 ){
```

```
        perror("fork");
```

```
        exit(1);
```

```
    }
```

```
    if(pid == 0){ //child
```

```
        glb++; var++;
```

```
    }
```

```
    else{ // parent
```

```
        sleep(2);
```

```
    }
```

```
    printf("pid = %d, glob = %d, var = %d\n",
```

```
    Return 0;
```

```
}
```

Process A (parent)

+-----+	
Code (text)	<- shared, read-only
+-----+	
Data (globals)	glb = 100
+-----+	
Heap	(unused here)
+-----+	
Stack	var = 88
+-----+	

Parent (PID 1234)

Child (PID 1235)

+-----+		+-----+	
Code (text)		Code (text)	<- same program
+-----+		+-----+	
Data (globals)	glb=100	Data (globals)	glb=100
+-----+		+-----+	
Heap		Heap	
+-----+		+-----+	
Stack	var=88	Stack	var=88
+-----+		+-----+	

Part III

Example 3: Files in `fork()`



The child has its own copy of the parent's file descriptors and share the same offset (`fork3.c`)

```
//fork3.c
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

#include <stdlib.h>

int main(){
    int pid, fd, i;
    char c;

    if ( (fd =open("test",O_RDWR|O_CREAT, 0644)) == -1 ){
        perror("test");
    }

    if( (pid = fork())< 0 ){
        perror("fork");
        exit(1);
    }
    if(pid == 0) // child
        for( i =65; i < 85; i++){ c
            = i; write(fd, &c ,1);
        }
    else // parent
        for( i = 0 ; i< 20; i++){ c
            = 64; // character @
            write(fd, &c ,1);
        }
    return 0;
}
```

Creating a new Process: `fork()`Terminating a process: `exit()``wait()` and `waitpid()`

The child has its own copy of the parent's file descriptors and share the same offset – Sequenced (`fork3_seq.c`)

Homework →

```
//fork3_seq.c
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

#include <stdlib.h>

int main(){
    int pid, fd, i;
    char c;

    if ( (fd = open("test", O_RDWR | O_CREAT, 0644)) == -1 ){
        perror("test");
    }

    if ( (pid = fork()) < 0 ){
        perror("fork");
        exit(1);
    }

    if (pid == 0) // child
        for( i = 65; i < 85; i++){ c
            = i; write(fd, &c, 1);
        }
    else // parent
        for( i = 0 ; i < 20; i++){ c
            = 64; // character @
            write(fd, &c, 1);
        }

    return 0;
}
```

Part III

Example 4: Multiple `fork()`



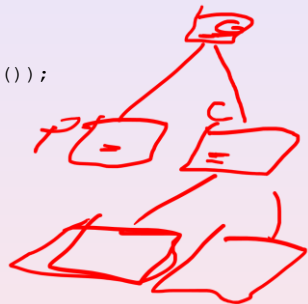
What is the output?

```
//q1.c
```

```
int main(){  
    printf("before fork, my pid is %d\n" , getpid());  
    fork();  
    fork();  
    fork();  
    printf("done, my pid is %d\n", getpid());  
}
```

```
//q2.c
```

```
int main(){  
    int i;  
    printf("before fork, my pid is %d\n" , getpid());  
    for (i=0; i<3; i++){  
        if ( fork()== 0 )  
            printf("Hi, I am child. My pid is %d\n", getpid());  
    }  
}
```



What is the output?

```
//q1.c
```

```
int main(){
```

```
→ printf("before fork, my pid is %d\n" , getpid());
```

```
fork();
```

```
fork();
```

```
fork();
```

```
printf("done, my pid is %d\n", getpid());
```

```
}
```

```
in
```

Process	fork()	fork()	fork()
P0			

```
}
```

What is the output?

```
//q1.c
```

```
int main(){
    printf("before fork, my pid is %d\n" , getpid());
    → fork();
    fork();
    fork();
    printf("done, my pid is %d\n", getpid());
}
```

Process	fork()	fork()	fork()
P0	P0 → P1		

What is the output?

//q1.c

```
int main(){  
    printf("before fork, my pid is %d\n" , getpid());  
    fork();  
    → fork();  
    fork();  
    printf("done, my pid is %d\n", getpid());  
}
```

Process	fork ()	fork ()	fork ()
P0	P0 → P1	P0 → P2	
		P1 → P3	

What is the output?

//q1.c

```
int main(){  
    printf("before fork, my pid is %d\n" , getpid());  
    fork();  
    fork();  
    → fork();  
    printf("done, my pid is %d\n", getpid());  
}
```

Process	fork ()	fork ()	fork ()
P0	P0 → P1	P0 → P2	P0 → P4
		P1 → P3	P1 → P5
			P2 → P6
			P3 → P7

What is the output?

//q1.c

```
int main(){
    printf("before fork, my pid is %d\n" , getpid());
    fork();
    fork();
    fork();
    printf("done, my pid is %d\n", getpid());
}
```

P0

//q2.c

```
int main(){
    int i;
    printf("before fork, my pid is %d\n" , getpid());
    for (i=0; i<3; i++){
        if (fork() == 0 )
            printf("Hi, I am child. My pid is %d\n", getpid());
    }
}
```

P1

P3

P7

P5

P2

P6

P4

What is the output?

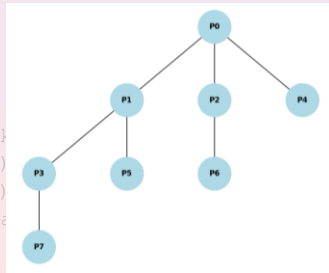
```
//q1.c
```

```
int main(){
    printf("before fork, my pid is %d\n" , getpid());
    fork();
    fork();
    fork();
    printf("done, my pid is %d\n", getpid());
}
```

P0

```
//q2.c
```

```
int main(){
    int i;
    for (i=0; i<3; i++){
        if (fork()== 0 )
            printf("Hi, I am %d\n", getpid());
    }
}
```



What is the output?

```
//q1.c
```

```
int main(){  
    printf("before fork, my pid is %d\n" , getpid());  
    fork();  
    fork();  
    fork();  
    printf("done, my pid is %d\n", getpid());  
}
```

```
//q2.c
```

```
int main(){  
    int i;  
    printf("before fork, my pid is %d\n" , getpid());  
    for (i=0; i<3; i++){  
        if ( fork()== 0 )  
            printf("Hi, I am child. My pid is %d\n", getpid());  
    }  
}
```

What is the output?

P0: `q1.c`

```
int main(){
    P1
    print("before fork, my pid is %d\n" , getpid());
    P3
    fork(); P7
    P5
    fork();
    P2
    fork();
    P6
    P4
    printf("done, my pid is %d\n", getpid());
}
```

//q2.c

```
int main(){
    int i;
    printf("before fork, my pid is %d\n" , getpid());
    for (i=0; i<3; i++){
        if ( fork()== 0 )
            printf("Hi, I am child. My pid is %d\n", getpid());
    }
}
```

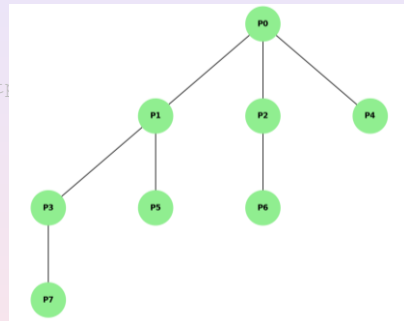
What is the output?

P0: `q2.c`

```
int main() {
    P1
    print("before fork, my pid is %d\n", getpid());
    P3
    fork(); P7
    P5
    fork();
    P2
    fork();
    P6
    P4
    printf("done, my pid is %d\n", getpid());
}
```

//q2.c

```
int main() {
    int i;
    printf("before fork, my pid is %d\n", getpid());
    for (i=0; i<3; i++){
        if ( fork()== 0 )
            printf("Hi, I am child. My pid is %d\n", getpid());
    }
}
```



Creating a new Process: `fork()`Terminating a process: `exit()``wait()` and `waitpid()`

What is the output?

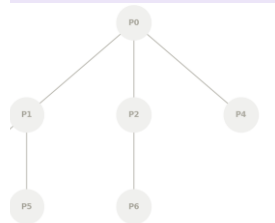
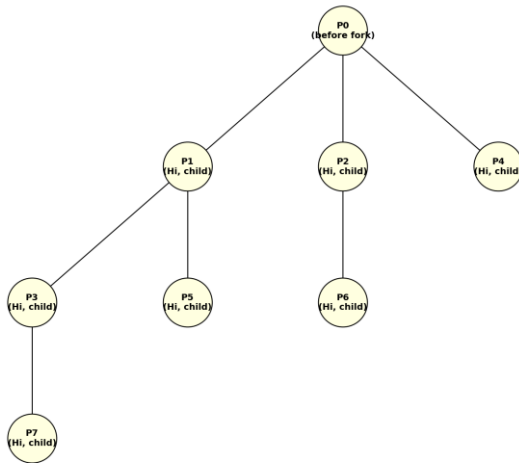
P0.c

```
int main()
{
    printf("P0\n");
    fork();
    fork();
    fork();
    printf("P0\n");
}
```

P1.c

```
int main()
{
    int i;
    printf("P1\n");
    for (i=0; i<10; i++)
    {
        if (i%2 == 0)
            printf("P1\n");
    }
}
```

Process Tree with Messages



Part IV

Example 5: Sequenced – Un-sequenced `fork()`



Creating a new Process: `fork()`Terminating a process: `exit()``wait()` and `waitpid()`

Child and Parent use terminal

Unsequenced (`tellwait1.c`)

```
//tellwait1.c
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

static void charatotime(char *);

int main(void)
{
    pid_t pid;

    if ((pid = fork()) < 0) {
        perror("fork error");
    } else if (pid == 0) {
        charatotime("output from child\n");
    } else {
        charatotime("output from parent\n");
    }
    exit(0);
}

static void charatotime(char *str)
{
    char *ptr;
    int c;

    setbuf(stdout, NULL); /* disable buffering */
    for (ptr = str; (c = *ptr++) != 0; )
        putc(c, stdout);
}
```

Creating a new Process: `fork()`Terminating a process: `exit()``wait()` and `waitpid()`

Child and Parent use terminal

Sequenced (tellwait2.c)

Homework →

```
//tellwait2.c
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

#include <stdlib.h>

int main(){
    int pid, fd, i;
    char c;

    if ( (fd = open("test", O_RDWR | O_CREAT, 0644)) == -1 ){
        perror("test");
    }

    if ( (pid = fork()) < 0 ){
        perror("fork");
        exit(1);
    }
    if (pid == 0) // child
        for( i = 65; i < 85; i++){ c
            = i; write(fd, &c, 1);
        }
    else // parent
        for( i = 0 ; i < 20; i++){ c
            = 64; // character @
            write(fd, &c, 1);
        }
    return 0;
}
```

Part V

Assignment #1: Overview



Assignment #1: Overview

- It's all about Coding!
- Reading methodology:
 - Read your examples
 - Access your textbook
- Work methodology:
 - Start as early as possible
 - Read carefully!
 - Read again!
 - Reach out for clarifications early
 - Make your example easily readable
- Submit way before the deadline
- Avoid using the grace period



Assignment #1: Overview

- It's all about coding
- Questions on file I/O:
 - Standard calls
 - System calls
 - Write program
 - Re-write program
- Make sure it compiles
- Make sure the logic is correct



Part VI

Attendance Quiz (Not graded)



Quiz



Attendance Quiz

701968234901768023405879312016
670980321768767



774862586034187069210783468019
701968234901768023405879312016
670980321768767

WPOYFXI LFJXI G I E
H I MPHLOM ZFOLI PG
BXNYSXWHYGI JLG I L
OLI EPC OOUJPL QDZC
NYDWIGL OUPJL QDZC

Thank you!
Questions?

DEFYXHQGRUOSGRP RN	3236343972474116
QJRYVQJXVRI DKUEVP	893833239594578
KGWBGQFHP RSUP RU	0310019701552750
XURNYUXZ DCSYUUM I	7875465400196596
WHMFSP UIQYSUQYRSHRG	5019353606937419

00110011100111
11111010000100
11000000001111
00111100111100
00101111001100
00000111011011
11001100110011

Dr. Andreas S. Maniatis

Assistant Professor

Andreas.Maniatis@UWindsor.ca

<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor